

Linguistic Pattern Recognition for MBTI Classification: A Data-Driven Decision Tree Analysis of Social Media Posts

Laavanya Sahay, Heer Desai, Saanvi Tyagi

April 2, 2026

1 Introduction

The Myers-Briggs Type Indicator (MBTI) is a personality classification system that categorizes individuals into sixteen distinct types based on four psychological dichotomies: Introversion vs. Extroversion (I/E), Sensing vs. Intuition (S/N), Thinking vs. Feeling (T/F), and Judging vs. Perceiving (J/P). Traditionally, this assessment relies on self-reported questionnaires, which are often subject to self-perception biases.

Social media activity provides a measurable record of behavior, including language patterns, engagement style, and expressive tendencies. These digital traces enable a computational approach to personality prediction based on observable data rather than subjective responses.

This project applies tree-based machine learning techniques to transform linguistic patterns into structured features and classify users into MBTI types.

Project Question/Goal: How can we create a modified version of the MBTI test that uses linguistic patterns in social media text posts to predict a user’s personality type through a data-driven decision tree?

2 Datasets

2.1 Reddit MBTI Dataset

- **Source:** Kaggle (Minhao Zhang, 2024)
- **Format:** CSV
- **Used Features:** `author`, `body`, `mbti`

Due to the large size of the dataset (approximately 3GB), we restricted our analysis to the first 1000 rows using terminal-based Kaggle tools. This preprocessing step reduced runtime and memory usage while preserving sufficient diversity for feature extraction.

2.2 Twitter Dataset

- **Format:** Multiple CSV files (`mbti_labels`, `user_info`, `user_tweets`)
- **Used Features:**
 - `mbti_labels`: `id`, `mbti_personality`
 - `user_info`: `id`, `average_tweet_length`
 - `user_tweets`: `id`, `tweet_1`, `tweet_2`, ..., `tweet_200`

2.3 Personality Cafe Dataset

- **Format:** CSV
- **Used Features:** type, posts

3 Computational Overview

3.1 Program Architecture

The program is divided into modular components:

- `filter_data.py`: Data ingestion and normalization into `User` objects
- `linguistic_scores.py`: Feature extraction and score computation
- `mbti_helper.py`: Decision tree splitting logic
- `main.py`: Tree construction and console interaction
- `visualisations.py`: Pygame-based graphical interface

Each module isolates a stage of the pipeline, allowing raw text data to be systematically transformed into structured inputs for classification.

3.2 Data Representation

The core data abstraction is the `User` class, which represents an individual as a vector of numerical features derived from their text data. Each user contains:

- an MBTI label (ground truth for training)
- average post length
- four continuous scores corresponding to MBTI dimensions:
 - `social_score` (E/I)
 - `complexity_score` (N/S)
 - `expressiveness_score` (F/T)
 - `structure_score` (J/P)

These values are computed directly from text using deterministic feature extraction functions, ensuring that all users across Reddit, Twitter, and other datasets are represented in a consistent format.

This structured representation is essential for tree-based learning, as each feature provides a dimension along which the dataset can be partitioned.

3.3 Feature Engineering

Raw text posts are processed using `get_linguistic_features`, which computes normalized densities of linguistic signals such as punctuation (e.g., exclamation and question marks, ellipses, formatting (capital letter density), social markers (hashtags, mentions, links, emojis) and lexical properties (vocab diversity and pronoun usage)

These low-level features are then aggregated into four higher-level scores via weighted functions such as `calculate_social_score` and `calculate_complexity_score`. The weighting system encodes assumptions about how linguistic behavior maps to personality traits (e.g., frequent tagging implies extroversion).

Importantly, this transformation produces continuous numerical values, enabling meaningful threshold-based splits in the decision tree.

Score Normalization

To ensure consistency across features, all computed scores are normalized to the range $[0.0, 1.0]$. Each scoring function defines a theoretical minimum and maximum based on its weighting scheme (e.g., maximum possible contribution from all linguistic features). The raw score is then scaled using min-max normalization:

$$\text{normalized} = \frac{\text{value} - \min}{\max - \min}$$

This normalization serves two key purposes. First, it ensures that all features operate on a common scale, preventing features with larger raw ranges from disproportionately influencing decision tree splits. Second, it aligns the feature space with the quiz input mapping, where user responses are also scaled between 0.0 and 1.0. This consistency is critical for maintaining coherence between the training data and interactive predictions.

3.4 Decision Tree Model

The central computational structure of the program is the `MBTITree`, a recursive binary decision tree trained on the processed dataset. A tree-based model is particularly well-suited for this task for two key reasons:

- **Interpretability:** Each decision corresponds to a human-understandable condition (e.g., `social_score < 0.12`), allowing the classification process to be traced step-by-step.
- **Compatibility with quiz interaction:** The hierarchical structure of a tree directly maps to a sequence of questions, making it possible to transform the model into an interactive system without additional abstraction.

The tree is constructed using the `build_mbti_tree` method, which recursively:

1. selects the best feature and threshold using `find_best_split`
2. partitions users into two groups based on that threshold
3. repeats the process on each subset until a stopping condition is met

Each node stores:

- a feature to split on
- a threshold value (typically derived from the dataset distribution)
- a count of MBTI labels reaching that node

Leaf nodes store the majority MBTI type, which becomes the final prediction.

3.5 User Interaction and Output

The trained decision tree is reused as an interactive quiz. This is enabled by the `predict` method, which traverses the tree based on user input rather than precomputed feature values.

The flow of interaction is as follows:

1. The program begins at the root of the tree.

2. At each node, the user is asked a question corresponding to the node’s feature (via `ask_user` or the Pygame UI).
3. The user responds on a Likert scale (1–5), which is mapped to a numerical value.
4. This value is compared against the node’s threshold:
 - if the value is less than the threshold, traversal continues to the left subtree
 - otherwise, traversal continues to the right subtree
5. This process repeats until a leaf node is reached.
6. The MBTI type stored at the leaf is returned as the prediction.

This design directly mirrors how the tree classifies training data, ensuring consistency between training and inference.

Ideally, the most accurate implementation would allow users to input their recent social media posts, enabling feature computation through the same pipeline used during training. However, this approach introduces both implementation complexity and privacy concerns. As a result, a quiz-based interface was chosen as a practical alternative that improves accessibility and user engagement.

Input Mapping and Threshold Design

User responses must be converted into numerical values to enable comparison with tree thresholds. Two mapping strategies were considered:

1. Uniform mapping (implemented):

$$1 \rightarrow 0.0, \quad 2 \rightarrow 0.25, \quad 3 \rightarrow 0.5, \quad 4 \rightarrow 0.75, \quad 5 \rightarrow 1.0$$

2. Threshold-relative mapping (alternative):

$$1 \rightarrow 0.0, \quad 2 \rightarrow \frac{t}{2}, \quad 3 \rightarrow t, \quad 4 \rightarrow t + \frac{1-t}{2}, \quad 5 \rightarrow 1.0$$

where t is the threshold at the current node.

The threshold-relative method aligns user input directly with the decision boundary at each node, ensuring that a neutral response corresponds exactly to the split point. While this could improve local decision accuracy, it introduces a critical drawback: the meaning of each response changes dynamically depending on the node. This reduces interpretability and makes the quiz experience inconsistent.

In contrast, the uniform mapping preserves a stable interpretation of responses across all questions. Although it may not perfectly align with every threshold, it ensures that the interaction remains intuitive and consistent. Given that usability is a primary goal of the quiz interface, the uniform mapping was selected.

3.6 Libraries

- **CSV**
- **Pygame**: Used to implement an event-driven graphical interface.
- **Emoji**: The function `emoji.emoji_count` is used to quantify emoji usage within text, contributing to the expressiveness score.

4 Running Instructions

1. **Setup:** `pip install pygame emoji`
2. **Datasets:** The zipped data folder (containing 5 files) can be accessed here: [Download Dataset Folder](#)
3. **Run:** you can either run in console (faster) by running `main.py` or run the visualization by running `visualization.py`
4. **Play:** Complete the 5 question quiz to receive your predicted MBTI.

5 Changes from Proposal

We shifted from a visualization-focused approach (Plotly) to an interactive decision-tree-driven quiz using Pygame. We also removed graph-based social network analysis due to inconsistency across datasets, instead focusing on linguistic feature extraction.

6 Discussion

Our project demonstrates that linguistic behavior can be translated into structured numerical features and used to approximate personality classification through a tree-based model. The decision tree provides an interpretable structure, allowing us to trace how specific behavioral features contribute to predictions.

However, several limitations affected performance. The median-based splitting strategy simplifies decision boundaries and may fail to capture complex relationships between features. Additionally, linguistic ambiguity, such as interpreting all-caps text as excitement, introduces noise into feature construction.

A major limitation was dataset imbalance. During testing, the model disproportionately predicted INFP and ENFP types. This was traced to significant overrepresentation of these types in all datasets, often by hundreds of samples. As a result, the tree learned biased decision boundaries.

Another limitation arises from the method of user input. While the quiz-based interface improves usability and accessibility, it introduces approximation error. The model is trained on precise linguistic features extracted directly from user-generated text, whereas quiz responses are subjective estimates of these behaviors. Ideally, users would input their recent social media posts, allowing the system to compute feature scores using the same pipeline as the training data. However, this approach raises practical concerns, including increased implementation complexity and user privacy. As a result, the quiz format represents a trade-off between accuracy and usability, favoring accessibility at the cost of precision.

Future improvements should directly address these limitations. To reduce bias in predictions, we would aim to construct more balanced datasets by ensuring a similar number of samples for each MBTI type rather than relying on naturally skewed distributions.

To address limitations in the decision tree itself, we could explore more flexible splitting strategies that better capture nuanced relationships between features, rather than relying solely on median-based thresholds. We would also refine how linguistic features are interpreted, particularly ambiguous signals such as all-caps text, by incorporating more contextual rules or additional indicators to better distinguish between emotional emphasis and structured formatting.

Finally, improving the user input process would be a key area of development. While maintaining a user-friendly interface, we could introduce optional text-based input (such as allowing users to paste recent posts) alongside the quiz. This would help bridge the gap between the model's training data and real-world usage, improving accuracy while still respecting user convenience and privacy.

Despite these challenges, the project successfully integrates non-trivial dataset processing, custom algorithm design, and interactive visualization. It highlights both the potential and limitations of applying computational models to human personality.

References

- “16Personalities.” *16Personalities*. NERIS Analytics Limited.
<https://www.16personalities.com/>
- Choong, EnJun. *8k MBTI Dataset From Personality Cafe*. 13 May 2021. figshare.
<https://doi.org/10.6084/m9.figshare.14587572>. Accessed 1 March 2026.
- Plotly Technologies Inc. *Collaborative Data Science*. 2015. Plotly, Montréal.
<https://plot.ly>. Accessed 1 March 2026.
- Rai, Sanket. *Twitter MBTI Personality Types Dataset*. Kaggle.
<https://www.kaggle.com/datasets/sanketrai/twitter-mbti-dataset>. Accessed 1 March 2026.
- Shinners, Pete. *Pygame Documentation*.
<https://www.pygame.org/docs/>. Accessed 1 March 2026.
- Zhang, Minhao. *Reddit MBTI Dataset*. 2024. Kaggle.
<https://www.kaggle.com/datasets/minhaozhang1/reddit-mbti-dataset>. Accessed 1 March 2026.